



35-MAVZU · 42 TADAN · IV BOB. PYTHON, POSTGRESQL VA DJANGO

Django Template tizimi va dinamik sahifalar.

Template va statik fayllar

render() va kontekst, Django Template Language: o'zgaruvchilar, {% if %}, {% for %}, filtrlar, {% url %}; shablon merosi (extends, block, include); CSS, JavaScript va rasmlar — static fayllar.

2 soat · 1 nazariy + 1 amaliy

Bu darsda nimani o'rganasiz

- HTML'ni view'dan ajratib, **shablon** (template) fayllarda saqlashni;
- View'dan shablonga ma'lumot uzatishni (`render` va kontekst);
- Django shablon tili: o'zgaruvchilar, shartlar, sikllar, filtrlar;
- **Shablon merosi** — umumiy sahifa "skeleti"ni bir marta yozishni;
- CSS, JavaScript va rasmlarni (**static fayllar**) ulashni.

Bu dars I-II boblardagi HTML/CSS/JS bilimlaringizni Django bilan birlashtiradi!

Template nima?

34-mavzuda HTML'ni Python satri ichida yozdik — noqulay va xavfli. **Template (shablon)** — oddiy HTML fayl, lekin unda "**bo'sh joylar**" bor, ularni Django ma'lumot bilan to'ldiradi.



O'xshatish: tabriknoma shabloni

Do'konda tayyor tabriknoma: "Hurmatli _____, sizni _____ bilan tabriklaymiz!" Siz faqat bo'sh joylarni to'ldirasiz. Shablon bir xil, ism va sabab har safar boshqa. Django template ham shunday: `<h1>{{ sarlavha }}</h1>` — har bir maqola uchun o'z sarlavhasi qo'yiladi.

Shablonlar papkasi va render()

- 1 Papkalar tuzilmasini yarating (ilova nomini **ikki marta** takrorlash — nomlar to'qnashmasligi uchun Django an'anasi):

Matn

```
blog/  
└─ templates/  
    └─ blog/  
        ├── bosh_sahifa.html  
        └─ maqola.html
```

- 2 View'da `render` ishlatish:

blog/views.py

```
from django.shortcuts import render  
from django.http import Http404  
  
MAQOLALAR = [ ... ] # 34-mavzudagi ro'yxat  
  
def bosh_sahifa(request):  
    kontekst = { # shablonga  
yuboriladigan ma'lumotlar (dict)  
        "sarlavha": "Maktab blogi",  
        "maqolalar": MAQOLALAR,  
    }  
    return render(request, "blog/bosh_sahifa.html", kontekst)  
  
def maqola(request, id):  
    for m in MAQOLALAR:  
        if m["id"] == id:  
            return render(request, "blog/maqola.html", {"maqola": m})  
    raise Http404("Maqola topilmadi")
```

`render(request, shablon_yo'li, kontekst)` — shablonni topadi, kontekstdagi ma'lumotlar bilan to'ldiradi va tayyor HTML javob qaytaradi.

Django Template Language (DTL)

Sintaksis	Vazifasi	Misol
<code>{{ ... }}</code>	O'zgaruvchini chiqarish	<code>{{ maqola.sarlavha }}</code>
<code>{% ... %}</code>	Teg — mantiq (shart, sikl, havola)	<code>{% for m in maqolalar %}</code>
<code>{{ x filtr }}</code>	Filtr — qiymatni o'zgartirish	<code>{{ ism upper }}</code>
<code>{# ... #}</code>	Izoh	<code>{# bu ko'rinmaydi #}</code>

O'zgaruvchilar

```
blog/templates/blog/maqola.html

<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <title>{{ maqola.sarlavha }}</title>
</head>
<body>
  <h1>{{ maqola.sarlavha }}</h1>           {# dict kaliti yoki obyekt atributi – nuqta bilan #}
  <p>Kategoriya: {{ maqola.kategoriya }}</p>
  <p>{{ maqola.matn }}</p>
  <a href="/">← Bosh sahifa</a>
</body>
</html>
```



Avtomatik xavfsizlik

Agar matnda `<script>` bo'lsa, Django uni **avtomatik ekranlaydi** (zararsiz matnga aylantiradi) — XSS hujumlaridan himoya (19-mavzuni eslang). 34-mavzudagi satr bilan yasalgan HTML'dan farqi aynan shu.

Shartlar: {% if %}

Django shablon

```
{% if maqolalar %}
    <p>Jami {{ maqolalar|length }} ta maqola</p>
{% else %}
    <p>Hozircha maqola yo'q 😞</p>
{% endif %}

{% if user.is_authenticated %}
    <p>Salom, {{ user.username }}!</p>
{% elif yosh ≥ 14 and sinf = 8 %}
    ...
{% endif %}
```

Sikllar: {% for %}

blog/templates/blog/bosh_sahifa.html

```
<h1>{{ sarlavha }}</h1>
<ul>
    {% for m in maqolalar %}
        <li>
            {{ forloop.counter }}.                {# 1, 2, 3 –
            tartib raqami #}
            <a href="{% url 'blog:maqola' m.id %}">{{ m.sarlavha }}</a>
            <small>({{ m.kategoriya }})</small>
        </li>
    {% empty %}
        <li>Maqolalar yo'q</li>                    {# ro'yxat bo'sh
bo'lsa #}
    {% endfor %}
</ul>
```

- `{% url 'blog:maqola' m.id %}` — 34-mavzudagi nomlangan URL'dan havola yasaydi. Manzillarni qo'lda yozmang!
- `forloop.counter` (1 dan), `forloop.first`, `forloop.last` — sikl haqida ma'lumot.



Shablonda faqat **ko'rsatish mantiqi** bo'lsin. Murakkab hisob-kitob, bazaga so'rov — view'ning ishi. DTL ataylab cheklangan: unda Python funksiyalarini to'g'ridan-to'g'ri chaqirib bo'lmaydi.

Filtrlar

Django shablon

<code>{{ ism upper }}</code>	→ ALI
<code>{{ ism lower }}</code>	→ ali
<code>{{ ism title }}</code>	→ Ali Valiyev
<code>{{ matn truncatewords:20 }}</code>	→ birinchi 20 so'z...
<code>{{ matn truncatechars:100 }}</code>	→ birinchi 100 belgi...
<code>{{ maqolalar length }}</code>	→ 3
<code>{{ sana date:"d.m.Y" }}</code>	→ 06.10.2026
<code>{{ sana date:"d E Y, H:i" }}</code>	→ 06 oktabr 2026, 15:30
<code>{{ narx floatformat:2 }}</code>	→ 12500.00
<code>{{ izoh default:"Izoh yo'q" }}</code>	→ qiymat bo'sh bo'lsa
<code>{{ matn linebreaks }}</code>	→ qator o'tishlarini <code><p></code> va <code>
</code> ga aylantiradi

Shablon merosi — eng muhim imkoniyat

Saytning har bir sahifasida bir xil header, menyu, footer va CSS bor. Ularni har bir shablonga nusxalash — xato. **Shablon merosi**: umumiy skelet bitta `base.html` da yoziladi, boshqa sahifalar faqat **o'zgaruvchi qismlarini** to'ldiradi.



O'xshatish: daftar varag'i

Maktab daftarining har bir varag'ida bir xil chiziqlar va hoshiya bor (`base.html`). Har bir varaqqa faqat o'sha kunning darsi yoziladi (block content). Hoshiyani har safar qayta chizmaysiz.

```

{% load static %}
<!DOCTYPE html>
<html lang="uz">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}Maktab blogi{% endblock %}</title>
  <link rel="stylesheet" href="{% static 'blog/css/style.css' %}">
</head>
<body>
  <header class="header">
    <a class="logo" href="{% url 'blog:bosh' %}">🏠 Maktab blogi</a>
    <nav>
      <a href="{% url 'blog:bosh' %}">Bosh sahifa</a>
      <a href="{% url 'blog:haqida' %}">Biz haqimizda</a>
    </nav>
  </header>

  <main class="konteyner">
    {% block content %}{% endblock %}          {# - har bir sahifa shu joyni
to'ldiradi #}
  </main>

  <footer class="footer">© 2026 5-maktab o'quvchilari</footer>
  <script src="{% static 'blog/js/main.js' %}"></script>
</body>
</html>

```

blog/templates/blog/bosh_sahifa.html

```
{% extends "blog/base.html" %}

{% block title %}Bosh sahifa – Maktab blogi{% endblock %}

{% block content %}
    <h1>So'nggi maqolalar</h1>
    <div class="kartalar">
        {% for m in maqolalar %}
            {% include "blog/_karta.html" %}
        {% empty %}
            <p>Hozircha maqola yo'q.</p>
        {% endfor %}
    </div>
{% endblock %}
```

blog/templates/blog/_karta.html

```
{# Qayta ishlatiladigan bo'lak: bitta maqola kartasi #}
<article class="karta">
    <span class="belgi">{{ m.kategoriya }}</span>
    <h3><a href="{% url 'blog:maqola' m.id %}">{{ m.sarlavha }}</a></h3>
    <p>{{ m.matn|truncatewords:15 }}</p>
</article>
```

blog/templates/blog/maqola.html

```
{% extends "blog/base.html" %}

{% block title %}{{ maqola.sarlavha }}{% endblock %}

{% block content %}
    <article>
        <h1>{{ maqola.sarlavha }}</h1>
        <p class="belgi">{{ maqola.kategoriya }}</p>
        {{ maqola.matn|linebreaks }}
        <a href="{% url 'blog:bosh' %}">← Barcha maqolalar</a>
    </article>
{% endblock %}
```

Teg	Vazifasi
<code>{% extends "base.html" %}</code>	Ota shablondan meros olish (faylning birinchi qatori bo'lishi shart)
<code>{% block nom %}...{% endblock %}</code>	O'zgaradigan joy (ota shablonda e'lon qilinadi, bola uni to'ldiradi)
<code>{% include "fayl.html" %}</code>	Boshqa shablon bo'lagini qo'shish (karta, menyu)

Static fayllar: CSS, JavaScript, rasmlar

CSS, JS va logotip kabi o'zgarmaydigan fayllar Django'da **static fayllar** deyiladi.

Matn

```
blog/
├── static/
│   ├── blog/
│   │   ├── css/style.css
│   │   ├── js/main.js
│   │   └── img/logo.png
```

Django shablon

```
{% load static %}                                {# shablon boshida
bir marta #}
<link rel="stylesheet" href="{% static 'blog/css/style.css' %}">

<script src="{% static 'blog/js/main.js' %}"></script>
```

config/settings.py

```
STATIC_URL = "static/" # brauzerdagi manzil boshlanishi
STATICFILES_DIRS = [BASE_DIR / "static"] # (ixtiyoriy) loyiha darajasidagi
umumiy static papka
STATIC_ROOT = BASE_DIR / "staticfiles" # serverga joylashda yig'iladigan
joy (41-mavzu)
```

blog/static/blog/css/style.css

```
* { box-sizing: border-box; }
body { margin: 0; font-family: "Segoe UI", Arial, sans-serif; background:
#f8fafc; color: #1f2937; }
.header { display: flex; justify-content: space-between; align-items: center;
padding: 16px 32px;
background: #4f46e5; }
.header a { color: white; text-decoration: none; margin-left: 18px; }
.logo { font-weight: 800; font-size: 1.3rem; margin-left: 0 !important; }
.konteyner { max-width: 1000px; margin: 30px auto; padding: 0 16px; min-height:
60vh; }
.kartalar { display: grid; grid-template-columns: repeat(auto-fill,
minmax(260px, 1fr)); gap: 20px; }
.karta { background: white; padding: 20px; border-radius: 14px; box-shadow: 0
6px 18px rgba(0,0,0,.06);
transition: transform .2s; }
.karta:hover { transform: translateY(-4px); }
.karta h3 a { color: #1e1b4b; text-decoration: none; }
.belgi { display: inline-block; background: #eef2ff; color: #4f46e5; padding:
2px 10px;
border-radius: 999px; font-size: 13px; font-weight: 600; }
.footer { text-align: center; padding: 24px; color: #6b7280; }
```



Media fayllar — foydalanuvchilar yuklaydigan fayllar (maqola rasmlari, avatarlar). Ular static'dan farq qiladi va `MEDIA_URL` / `MEDIA_ROOT` bilan sozlanadi — 36-mavzuda `ImageField` bilan birga ko'ramiz.



Static fayl o'zgarishi ko'rinmasa — brauzer eski nusxani keshlagan. `Ctrl` + `F5` bilan sahifani to'liq yangilang. Yangi `static` papka qo'shganda serverni qayta ishga tushiring.

Amaliy mashg'ulot

- 1 `base.html`, `bosh_sahifa.html`, `_karta.html`, `maqola.html` shablonlarini yarating va view'larni `render` ga o'tkazing.
- 2 Static papkada `style.css` yarating va ulang. Sahifa 8-mavzudagi portfolio kabi chiroyli ko'rinishiga erishing.
- 3 `haqida.html` sahifasini `base.html` dan meros olib yarating.
- 4 `main.js` da kichik interaktivlik qo'shing: "🌙" tugmasi bosilganda tungi rejim (19-mavzu).
- 5 Kategoriya sahifasini (`kategoriya.html`) yarating: sarlavha `{{ nomi|title }}` va shu kategoriyadagi maqolalar (`_karta.html` ni `include` qilib).
- 6 Bosh sahifada maqolalar sonini ko'rsating va oxirgi kartadan keyin chiziq chizilmasin (`forloop.last`).

Ko'p uchraydigan xatolar

- ❌ `TemplateDoesNotExist` — shablon yo'li noto'g'ri yoki papka nomi `template` (s harfisiz). To'g'risi: `blog/templates/blog/fayl.html`.
- ❌ Invalid block tag 'static' — shablon boshida `{% load static %}` yo'q.
- ❌ `{% endfor %}`, `{% endif %}`, `{% endblock %}` ni unutish — `TemplateSyntaxError`.
- ❌ `{% extends %}` birinchi qatorda emas.
- ❌ `{{ maqola[sarlavha] }}` — DTL'da kvadrat qavs yo'q, faqat nuqta: `{{ maqola.sarlavha }}`.

O'zingizni tekshiring

? Shablonda o'zgaruvchi qiymatini chiqarish sintaksisi?

- A. `{% ism %}`
- B. `{{ ism }}`
- C. `${ism}`
- D. `<?= ism ?>`

? View'dan shablonga ma'lumot qanday uzatiladi?

- A. Global o'zgaruvchi orqali
- B. URL orqali
- C. `render(request, "shablon.html", kontekst_dict)`
- D. localStorage orqali

? Umumiy skeletdan meros olish uchun qaysi teg?

- A. `{% extends "base.html" %}`
- B. `{% include "base.html" %}`
- C. `{% import base %}`
- D. `{% inherit base %}`

? **Shablonda CSS faylni to'g'ri ulash?**

- A. `href="style.css"`
- B. `href="/blog/style.css"`
- C. `href="{{ static.css }}"`
- D. `href="{% static 'blog/css/style.css' %}"`

? `{{ matn|truncatewords:10 }}` nima qiladi?

- A. Matnni 10 marta takrorlaydi
- B. Matnning birinchi 10 so'zini ko'rsatadi
- C. 10 ta belgini o'chiradi
- D. Matnni 10 ga bo'ladi

? **Sikl ichida ro'yxat bo'sh bo'lgandagi holat uchun qaysi teg?**

- A. `{% else %}`
- B. `{% none %}`
- C. `{% empty %}`
- D. `{% null %}`



Uyga vazifa

- Blog dizaynini to'liq responsiv qiling (10-mavzu): telefonda bitta ustun, burger menyu.
- Maxsus 404 sahifa: `templates/404.html` yarating (u `DEBUG=False` bo'lganda ko'rinadi — 41-mavzuda tekshiramiz).

3. `pages` ilovasidagi sahifalarni ham umumiy `base.html` dan meros oladigan qiling (Maslahat: `config/settings.py` → `TEMPLATES[0]["DIRS"] = [BASE_DIR / "templates"]` va loyiha darajasidagi `templates/base.html`).

✓ Yodda tuting

- Shablonlar: `ilova/templates/ilova/*.html`; view'da `render(request, "ilova/fayl.html", {...})`.
- DTL: `{{ o'zgaruvchi }}`, `{% teg %}`, `{{ x|filtr }}`; `{% if %}`, `{% for %}`...`{% empty %}`, `{% url 'app:nom' arg %}`.
- Meros: `base.html` da `{% block content %}`; bolada `{% extends %}` + `{% block %}`; bo'laklar — `{% include %}`.
- Static: `ilova/static/ilova/...`, shablonda `{% load static %}` + `{% static 'yo'l' %}`.
- Django matnni avtomatik ekranlaydi — XSS'dan himoya.

□ Darsni o'qib bo'ldim

✓ Test javoblari

1. Shablonda o'zgaruvchi qiymatini chiqarish sintaksisi? — **B) {{ ism }}**
2. View'dan shablonga ma'lumot qanday uzatiladi? — **C) render(request, "shablon.html", kontekst_dict)**
3. Umumiy skeletdan meros olish uchun qaysi teg? — **A) {% extends "base.html" %}**
4. Shablonda CSS faylni to'g'ri ulash? — **D) href="{% static 'blog/css/style.css' %}"**
5. {{ matn|truncatewords:10 }} nima qiladi? — **B) Matnning birinchi 10 so'zini ko'rsatadi**
6. Sikl ichida ro'yxat bo'sh bo'lgandagi holat uchun qaysi teg? — **C) {% empty %}**